glmCH2_clean

Load and clean data

```
#load packages
library(tidyverse) #data manipulation, visualization etc. (dplyr, ggplot2, tidyr)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(DHARMa) #diagnostic tools for mixed regression models
```

```
## This is DHARMa 0.4.7. For overview type '?DHARMa'. For recent changes, type news(package = 'DHARMa')
```

```
library(glmmTMB) #fits generalized linear mixed models, including zero-inflated and overdispersed models
library(car) #tools for regression diagnostics and hypothesis tests
```

```
## Loading required package: carData
##
## Attaching package: 'car'
##
## The following object is masked from 'package:dplyr':
##
##     recode
##
## The following object is masked from 'package:purrr':
##
##     some
```

```
library(emmeans)   # for estimated marginal means (predictions)
```

```
## Welcome to emmeans.
## Caution: You lose important information if you filter this package's results.
## See '? untidy'
```

```r
#read in data
data <- read.csv("06_cleandata/combined_plot_data.csv")

#change character columns to factors
surveys <- data %>%
   mutate(
     site_id = as.factor(site_id),
     round = as.factor(round),
     plot_id = as.factor(plot_id),
     bee_date = as.Date(bee_date),
     bee_season_type = as.factor(bee_season_type),
     area = as.factor(area),
     cover_type = as.factor(cover_type))

#add plot type column and make it a factor
surveys <- surveys %>%
  mutate(plot_type = if_else(plot_id == "d", "ditch", "field"))
surveys <- surveys %>%
   mutate(
     plot_type = as.factor(plot_type))

#scale continuous predictors
surveys$julian.date_scaled <- as.numeric(scale(surveys$julian.date))
surveys$bare_perc_scaled <- as.numeric(scale(surveys$bare_perc))
surveys$grass_perc_scaled <- as.numeric(scale(surveys$grass_perc))
surveys$forb_perc_scaled <- as.numeric(scale(surveys$forb_perc))
surveys$grass_forb_perc_scaled <- as.numeric(scale(surveys$grass_forb_perc))
surveys$grass_h_scaled <- as.numeric(scale(surveys$grass_h))
surveys$forb_h_scaled <- as.numeric(scale(surveys$forb_h))
surveys$floral_perc_scaled <- as.numeric(scale(surveys$floral_perc))
surveys$grass_forb_floral_perc_scaled <- as.numeric(scale(surveys$grass_forb_floral_perc))
surveys$bombus_floral_perc_scaled <- as.numeric(scale(surveys$bombus_floral_perc))
surveys$flood_perc_scaled <- as.numeric(scale(surveys$flood_perc))
surveys$dry_root_perc_scaled <- as.numeric(scale(surveys$dry_root_perc))
surveys$bryo_perc_scaled <- as.numeric(scale(surveys$bryo_perc))
surveys$bare_flood_dry_bryo_perc_scaled <- as.numeric(scale(surveys$bare_flood_dry_bryo_perc))
surveys$floral_richness_scaled <- as.numeric(scale(surveys$floral_richness))
surveys$bombus_floral_richness_scaled <- as.numeric(scale(surveys$bombus_floral_richness))
surveys$bombus_floral_abun_scaled <- as.numeric(scale(surveys$bombus_floral_abun))
surveys$nest_searching_queens_scaled <- as.numeric(scale(surveys$nest_searching_queens))
surveys$workers_scaled <- as.numeric(scale(surveys$workers))
surveys$forage_vaco_scaled <- as.numeric(scale(surveys$forage_vaco))
surveys$num_burrows_scaled <- as.numeric(scale(surveys$num_burrows))
surveys$num_inactive_burrows_scaled <- as.numeric(scale(surveys$num_inactive_burrows))
surveys$num_active_dm_scaled <- as.numeric(scale(surveys$num_active_dm))
surveys$num_dm_size_scaled <- as.numeric(scale(surveys$num_dm_size))
surveys$num_inactive_dm_scaled <- as.numeric(scale(surveys$num_inactive_dm))
```

Choosing which vegetation variables to include as predictors

1. Use PCA to determine which vegetation variables to include in my models. I don't want to include all of the predictors, since some will be correlated with each other and therefore dilute their effects

of the response variables. I am using PCA so I can pick one variable that drives each of the main 3 components, and use these as the predictors in my model.

```
pca_res <- prcomp(surveys[, c("bare_perc_scaled",
                              "grass_forb_perc_scaled",
                              "grass_h_scaled",
                              "forb_h_scaled",
                              "bombus_floral_perc_scaled",
                              "flood_perc_scaled",
                              "dry_root_perc_scaled",
                              "bombus_floral_richness_scaled",
                              "bombus_floral_abun_scaled")], scale. = TRUE)
summary(pca_res)
```

```
## Importance of components:
##                           PC1    PC2    PC3     PC4     PC5     PC6     PC7
## Standard deviation     1.8037 1.2128 1.0888 0.94326 0.90070 0.82536 0.54028
## Proportion of Variance 0.3615 0.1634 0.1317 0.09886 0.09014 0.07569 0.03243
## Cumulative Proportion  0.3615 0.5249 0.6566 0.75551 0.84565 0.92135 0.95378
##                           PC8     PC9
## Standard deviation     0.5187 0.38329
## Proportion of Variance 0.0299 0.01632
## Cumulative Proportion  0.9837 1.00000
```

```
#see which variable drives each of the principle components
print(pca_res$rotation)
```

```
##                                    PC1         PC2        PC3          PC4
## bare_perc_scaled               0.24100694  0.51132677  0.1480093 -0.506337915
## grass_forb_perc_scaled        -0.42303293 -0.22390273 -0.3007926 -0.006568266
## grass_h_scaled                -0.49812518 -0.11060633 -0.0470433 -0.149976271
## forb_h_scaled                 -0.48569527 -0.08315431  0.1134057 -0.240186492
## bombus_floral_perc_scaled     -0.18883655  0.40010344 -0.1653710  0.651841412
## flood_perc_scaled              0.12306241 -0.07874923 -0.6378149  0.019972489
## dry_root_perc_scaled           0.02157359 -0.40268963  0.6227065  0.317142224
## bombus_floral_richness_scaled -0.43081001  0.26087490  0.1437940 -0.233788645
## bombus_floral_abun_scaled     -0.20509491  0.52226932  0.1730539  0.287860892
##                                       PC5          PC6         PC7         PC8
## bare_perc_scaled               0.06538925  0.370121563  0.49842251  0.10038801
## grass_forb_perc_scaled        -0.27778537 -0.153122311  0.54910763  0.48948701
## grass_h_scaled                 0.02851587  0.203518533  0.20705746 -0.38586034
## forb_h_scaled                  0.15759445  0.172355381 -0.08782852 -0.41865475
## bombus_floral_perc_scaled     -0.13014055  0.571974934 -0.03009873  0.03525875
## flood_perc_scaled              0.74841113  0.058657287  0.08301863  0.01827233
## dry_root_perc_scaled           0.42325486  0.207845797  0.28968027  0.20646515
## bombus_floral_richness_scaled  0.27097277  0.003296897 -0.48095986  0.58651301
## bombus_floral_abun_scaled      0.25156245 -0.628196361  0.27646719 -0.19483913
##                                       PC9
## bare_perc_scaled              -0.049064111
## grass_forb_perc_scaled        -0.196630307
## grass_h_scaled                 0.693496680
## forb_h_scaled                 -0.670154480
## bombus_floral_perc_scaled     -0.075975027
```

```
## flood_perc_scaled             -0.025721397
## dry_root_perc_scaled           0.012084553
## bombus_floral_richness_scaled  0.149185034
## bombus_floral_abun_scaled     -0.006968346
```

- PCA Results:
  - PC1: grass_h_scaled (-0.50),forb_h_scaled (-0.48),grass_forb_perc_scaled (-0.42),bombus_floral_richness_scal (-0.43),bombus_floral_abun_scaled (-0.22)
  - PC2: bare_perc_scaled (0.51),bombus_floral_abun_scaled (0.52),bombus_floral_perc_scaled (0.40), dry_root_perc_scaled (-0.40)
- Based on these PCA results and my research question, I am selecting to include the following predictors in my model:
  - grass_forb_perc_scaled (loads strongly on PCA1)
  - bombus_floral_abun_scaled (loads strongly on PCA1 &2)

3. Check pairwise correlations between this reduced set of variables

- correlation among all the pairs are all near zero –> predictors are all sufficiently independent! (R«0.7)

```
cor(surveys[, c("grass_forb_perc_scaled", "bombus_floral_abun_scaled")])
```

```
##                           grass_forb_perc_scaled bombus_floral_abun_scaled
## grass_forb_perc_scaled                1.00000000                0.07455522
## bombus_floral_abun_scaled             0.07455522                1.00000000
```

Nest-searching queens vs vegetation predictors

Plot model with interaction between vegetation variables and plot type to investigate whether the effects of vegetation cover and floral abundance on queen abundance/presence change depending on plot type (ditch vs field).

Steps

1) Try different random effect structures.

- Model without random effect fit data the best.

```
nsq_glm_veg_full <- glmmTMB(nest_searching_queens ~ grass_forb_perc_scaled*plot_type+
                              bombus_floral_abun_scaled*plot_type +
                            bee_season_type +
                            (1|area/site_id/plot_id),
                          ziformula = ~grass_forb_perc_scaled*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                            bee_season_type +
                            (1|area/site_id/plot_id),  # zero-inflation model
                          family = poisson,
                          data = surveys)
```

```
## Warning in finalizeTMB(TMBStruc, obj, fit, h, data.tmb.old): Model convergence
## problem; non-positive-definite Hessian matrix. See vignette('troubleshooting')
```

```
nsq_glm_veg_noArea <- glmmTMB(nest_searching_queens ~ grass_forb_perc_scaled*plot_type +
                                bombus_floral_abun_scaled*plot_type+
                              bee_season_type+
                              (1|site_id/plot_id),
                            ziformula = ~grass_forb_perc_scaled*plot_type +
                                bombus_floral_abun_scaled*plot_type +
                              bee_season_type +
                              (1|site_id/plot_id),  # zero-inflation model
                            family = poisson,
                            data = surveys)

nsq_glm_veg_justSite <- glmmTMB(nest_searching_queens ~ grass_forb_perc_scaled*plot_type +
                                bombus_floral_abun_scaled*plot_type +
                              bee_season_type+
                              (1|site_id),
                            ziformula = ~grass_forb_perc_scaled*plot_type +
                                bombus_floral_abun_scaled*plot_type+
                              bee_season_type +
                              (1|site_id),  # zero-inflation model
                            family = poisson,
                            data = surveys)

nsq_glm_veg_noRE <- glmmTMB(nest_searching_queens ~ grass_forb_perc_scaled*plot_type +
                                bombus_floral_abun_scaled*plot_type +
                              bee_season_type,
                            ziformula = ~grass_forb_perc_scaled*plot_type +
                                bombus_floral_abun_scaled*plot_type +
                              bee_season_type,  # zero-inflation model
                            family = poisson,
                            data = surveys)

#compare models
anova(nsq_glm_veg_noRE, nsq_glm_veg_justSite, nsq_glm_veg_noArea, nsq_glm_veg_full)
```

```
## Data: surveys
## Models:
## nsq_glm_veg_noRE: nest_searching_queens ~ grass_forb_perc_scaled * plot_type + , zi=~grass_forb_perc_
## nsq_glm_veg_noRE:     bombus_floral_abun_scaled * plot_type + bee_season_type, zi=    plot_type + bee
## nsq_glm_veg_justSite: nest_searching_queens ~ grass_forb_perc_scaled * plot_type + , zi=~grass_forb_p
## nsq_glm_veg_justSite:     bombus_floral_abun_scaled * plot_type + bee_season_type + , zi=    plot_typ
## nsq_glm_veg_justSite:     (1 | site_id), zi=~grass_forb_perc_scaled * plot_type + bombus_floral_abun_
## nsq_glm_veg_noArea: nest_searching_queens ~ grass_forb_perc_scaled * plot_type + , zi=    plot_type +
## nsq_glm_veg_noArea:     bombus_floral_abun_scaled * plot_type + bee_season_type + , zi=~grass_forb_pe
## nsq_glm_veg_noArea:     (1 | site_id/plot_id), zi=    plot_type + bee_season_type + (1 | area/site_id
## nsq_glm_veg_full: nest_searching_queens ~ grass_forb_perc_scaled * plot_type + , zi=~grass_forb_perc_
## nsq_glm_veg_full:     bombus_floral_abun_scaled * plot_type + bee_season_type + , zi=    plot_type +
## nsq_glm_veg_full:     (1 | area/site_id/plot_id), zi=~grass_forb_perc_scaled * plot_type + bombus_flo
##                      Df    AIC    BIC  logLik deviance  Chisq Chi Df Pr(>Chisq)
## nsq_glm_veg_noRE     14 297.27 344.13 -134.63   269.27
## nsq_glm_veg_justSite 16 301.03 354.59 -134.52   269.03 0.2332      2     0.8899
## nsq_glm_veg_noArea   18 304.17 364.42 -134.08   268.17 0.8661      2     0.6485
```

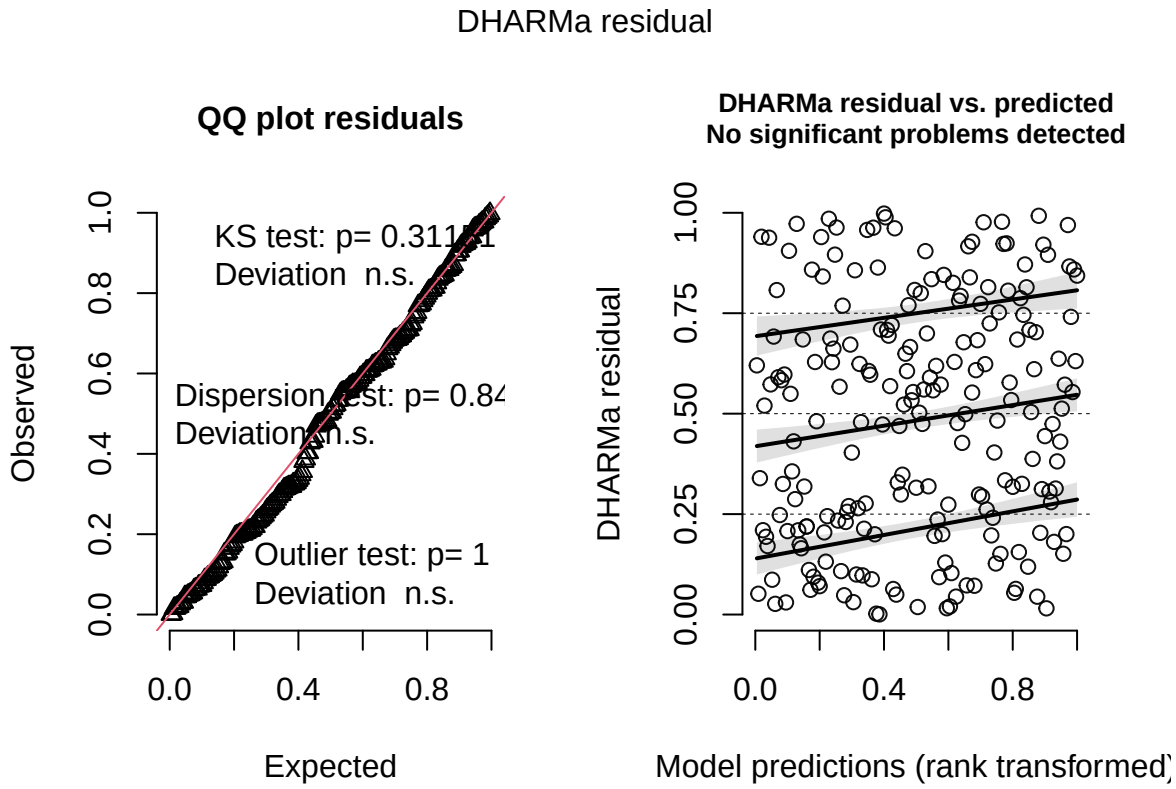
```
## nsq_glm_veg_full     20                                        2
```

```
##no random effect model is best, followed by just site model
```

2) Check diagnostics of model

- look good

```
#simulate residuals
residuals_nsq_glm_veg_noRE <- simulateResiduals(fittedModel = nsq_glm_veg_noRE, plot = TRUE)
```



3) Plot results

```
summary(nsq_glm_veg_noRE)
```

```
##  Family: poisson  ( log )
## Formula:          nest_searching_queens ~ grass_forb_perc_scaled * plot_type +
##     bombus_floral_abun_scaled * plot_type + bee_season_type
## Zero inflation:
## ~grass_forb_perc_scaled * plot_type + bombus_floral_abun_scaled *
##     plot_type + bee_season_type
## Data: surveys
##
```

```
##       AIC       BIC    logLik -2*log(L)  df.resid
##     297.3     344.1    -134.6     269.3       196
##
##
## Conditional model:
##                                        Estimate Std. Error z value Pr(>|z|)
## (Intercept)                              0.6100     0.2799   2.180   0.0293 *
## grass_forb_perc_scaled                   0.2642     0.1591   1.660   0.0969 .
## plot_typefield                          -0.5934     0.3963  -1.498   0.1343
## bombus_floral_abun_scaled               -0.1792     0.1620  -1.106   0.2688
## bee_season_typeworker                   -0.5874     0.6913  -0.850   0.3954
## grass_forb_perc_scaled:plot_typefield   -0.2839     0.2954  -0.961   0.3364
## plot_typefield:bombus_floral_abun_scaled  0.1519     0.3055   0.497   0.6190
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                                        Estimate Std. Error z value Pr(>|z|)
## (Intercept)                            -0.48423    0.56613  -0.855  0.39237
## grass_forb_perc_scaled                 -0.09105    0.35532  -0.256  0.79776
## plot_typefield                          0.57131    0.70218   0.814  0.41586
## bombus_floral_abun_scaled               0.07247    0.40543   0.179  0.85814
## bee_season_typeworker                   2.54386    0.79741   3.190  0.00142
## grass_forb_perc_scaled:plot_typefield  -0.99003    0.63409  -1.561  0.11844
## plot_typefield:bombus_floral_abun_scaled -0.79133  0.59773  -1.324  0.18554
##
## (Intercept)
## grass_forb_perc_scaled
## plot_typefield
## bombus_floral_abun_scaled
## bee_season_typeworker                  **
## grass_forb_perc_scaled:plot_typefield
## plot_typefield:bombus_floral_abun_scaled
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Plots

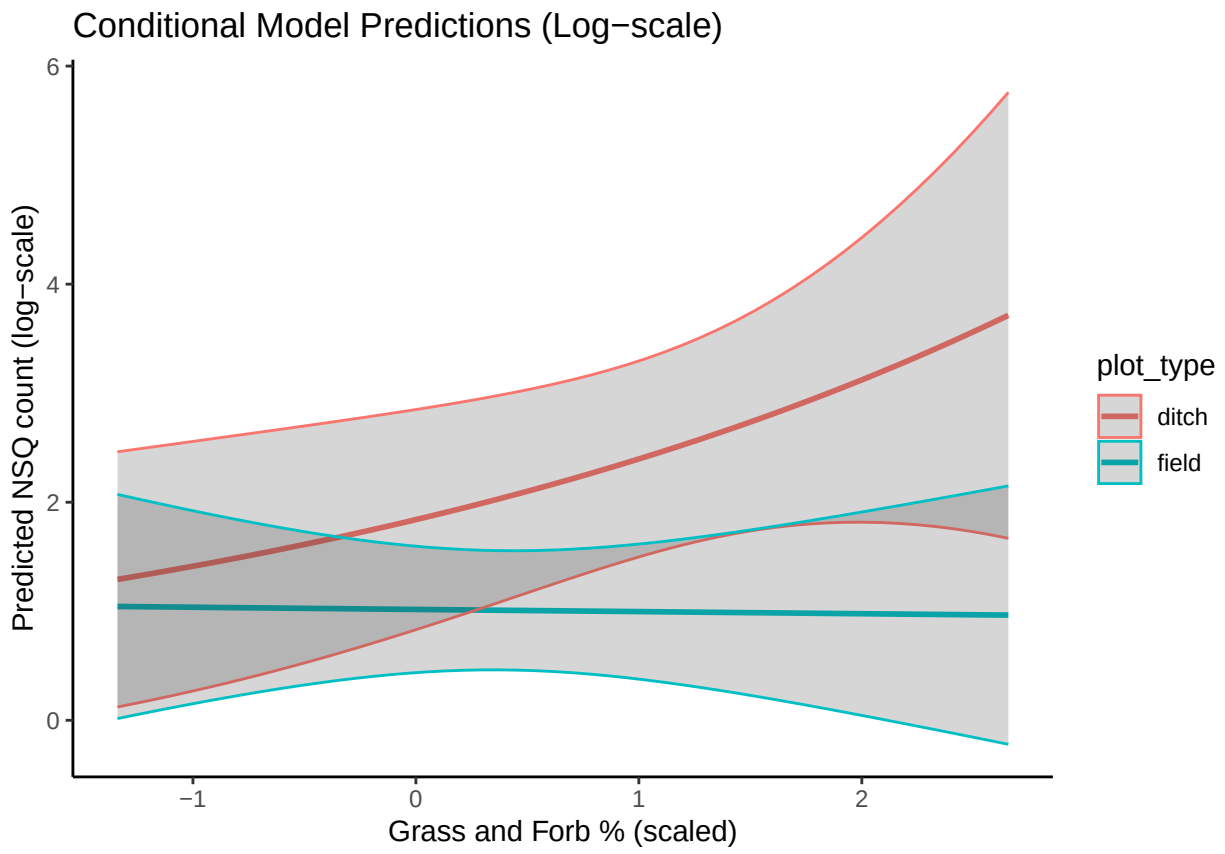
Grass and forb % cover

Conditional model:

```r
# Define a sequence of values for grass_forb_perc_scaled across its observed range
grass_seq <- seq(
  from = min(surveys$grass_forb_perc_scaled, na.rm = TRUE),
  to = max(surveys$grass_forb_perc_scaled, na.rm = TRUE),
  length.out = 50
)

#create new data gri
nsq_data_veg<- expand.grid(
  grass_forb_perc_scaled = grass_seq,
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
```

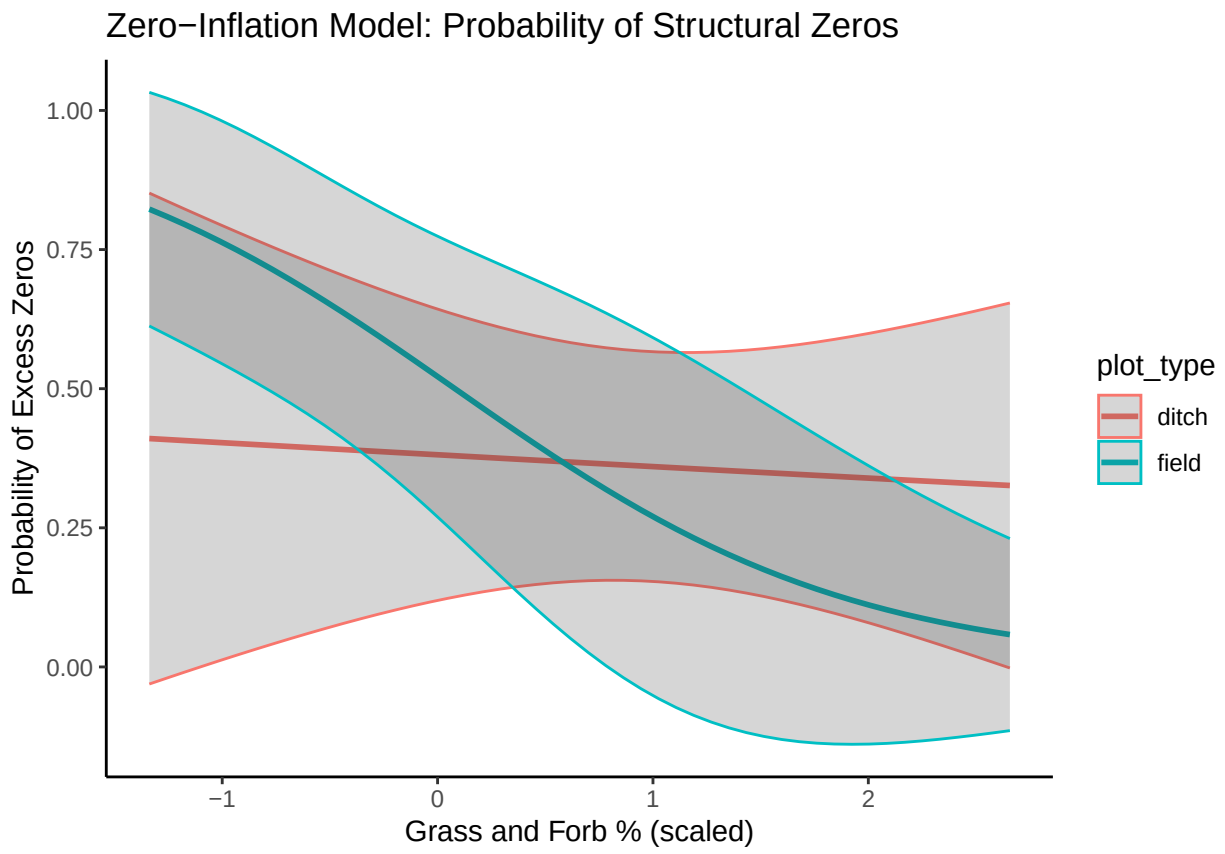
```
  bee_season_type = "nestsearching")


# Get predicted conditional probabilities and standard errors
pred_nsq_veg_cond <- predict(nsq_glm_veg_noRE,
                             newdata = nsq_data_veg,
                             type = "conditional",
                             se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_rodent_df <- cbind(nsq_data_veg,
                 cond_fit = pred_nsq_veg_cond$fit,
                 cond_se = pred_nsq_veg_cond$se.fit)

ggplot(pred_nsq_rodent_df, aes(x = grass_forb_perc_scaled, y = cond_fit, color = plot_type)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = cond_fit - 1.96*cond_se, ymax = cond_fit + 1.96*cond_se), alpha = 0.2) +
  labs(title = "Conditional Model Predictions (Log-scale)",
       x = "Grass and Forb % (scaled)",
       y = "Predicted NSQ count (log-scale)") +
  theme_classic()
```

```
## Warning: Using ‘size‘ aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use ‘linewidth‘ instead.
## This warning is displayed once every 8 hours.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```



Zero-inflated model:

```
# Get predicted zero probabilities and standard errors
pred_nsq_veg_zi <- predict(nsq_glm_veg_noRE,
                           newdata = nsq_data_veg,
                           type = "zprob",
                           se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_veg_df <- cbind(nsq_data_veg,
                 zi_fit = pred_nsq_veg_zi$fit,
                 zi_se = pred_nsq_veg_zi$se.fit)

ggplot(pred_nsq_veg_df, aes(x = grass_forb_perc_scaled, y = zi_fit, color = plot_type)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = zi_fit - 1.96*zi_se, ymax = zi_fit + 1.96*zi_se), alpha = 0.2) +
  labs(title = "Zero-Inflation Model: Probability of Structural Zeros",
       x = "Grass and Forb % (scaled)",
       y = "Probability of Excess Zeros") +
  theme_classic()
```

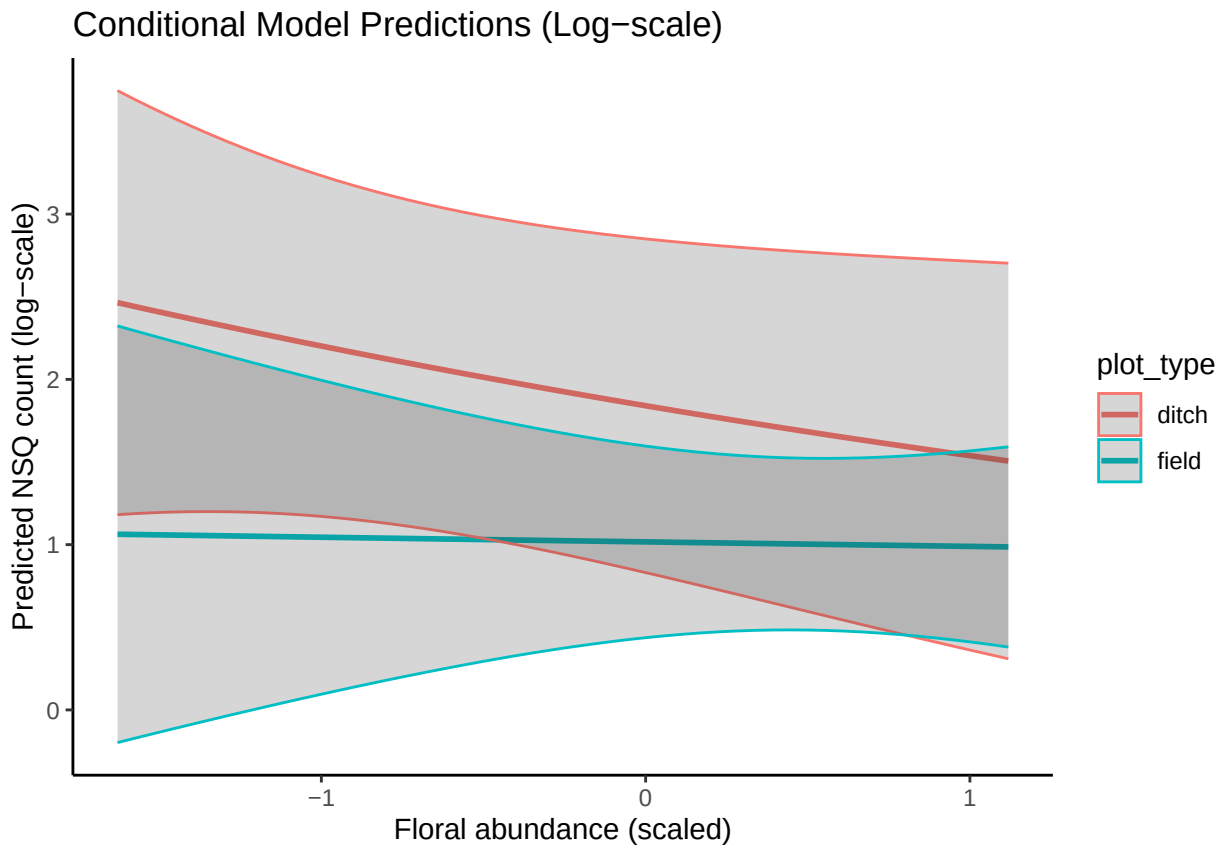


Floral abundance

Conditional model:

```
# Define a sequence of values for grass_forb_perc_scaled across its observed range
floral_seq <- seq(
  from = min(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
  to = max(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
  length.out = 50
)

#create new data gri
nsq_data_floral<- expand.grid(
  grass_forb_perc_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = floral_seq,
  bee_season_type = "nestsearching"
)

# Get predicted conditional probabilities and standard errors
pred_nsq_floral_cond <- predict(nsq_glm_veg_noRE,
                            newdata = nsq_data_floral,
                            type = "conditional",
                            se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_floral_df <- cbind(nsq_data_floral,
```

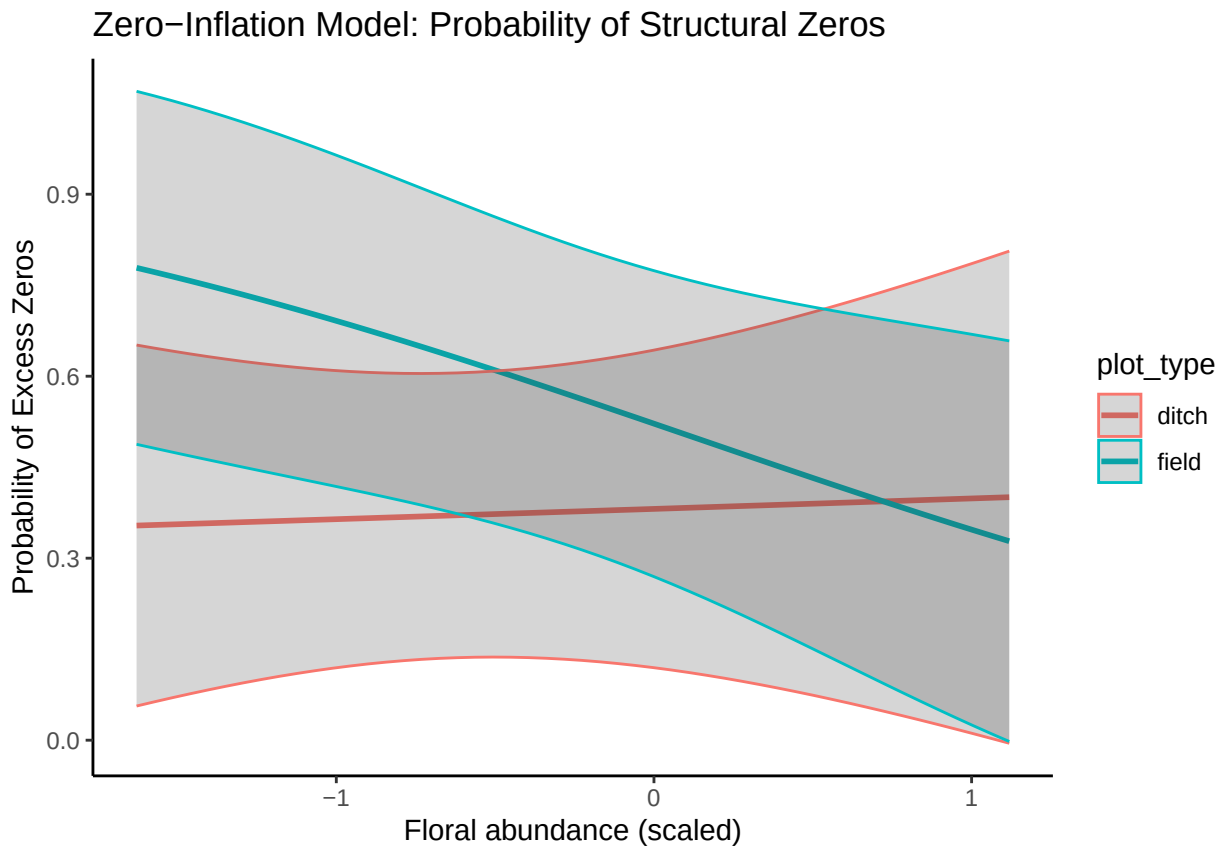
```
                cond_fit = pred_nsq_floral_cond$fit,
                cond_se = pred_nsq_floral_cond$se.fit)

ggplot(pred_nsq_floral_df, aes(x = bombus_floral_abun_scaled, y = cond_fit, color = plot_type)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = cond_fit - 1.96*cond_se, ymax = cond_fit + 1.96*cond_se), alpha = 0.2) +
  labs(title = "Conditional Model Predictions (Log-scale)",
       x = "Floral abundance (scaled)",
       y = "Predicted NSQ count (log-scale)") +
  theme_classic()
```



Zero-inflated model:

```
# Get predicted zero probabilities and standard errors
pred_nsq_floral_zi <- predict(nsq_glm_veg_noRE,
                              newdata = nsq_data_floral,
                              type = "zprob",
                              se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_floral_df <- cbind(nsq_data_floral,
                zi_fit = pred_nsq_floral_zi$fit,
                zi_se = pred_nsq_floral_zi$se.fit)

ggplot(pred_nsq_floral_df, aes(x = bombus_floral_abun_scaled, y = zi_fit, color = plot_type)) +
  geom_line(size = 1) +
```

```
  geom_ribbon(aes(ymin = zi_fit - 1.96*zi_se, ymax = zi_fit + 1.96*zi_se), alpha = 0.2) +
  labs(title = "Zero-Inflation Model: Probability of Structural Zeros",
       x = "Floral abundance (scaled)",
       y = "Probability of Excess Zeros") +
  theme_classic()
```

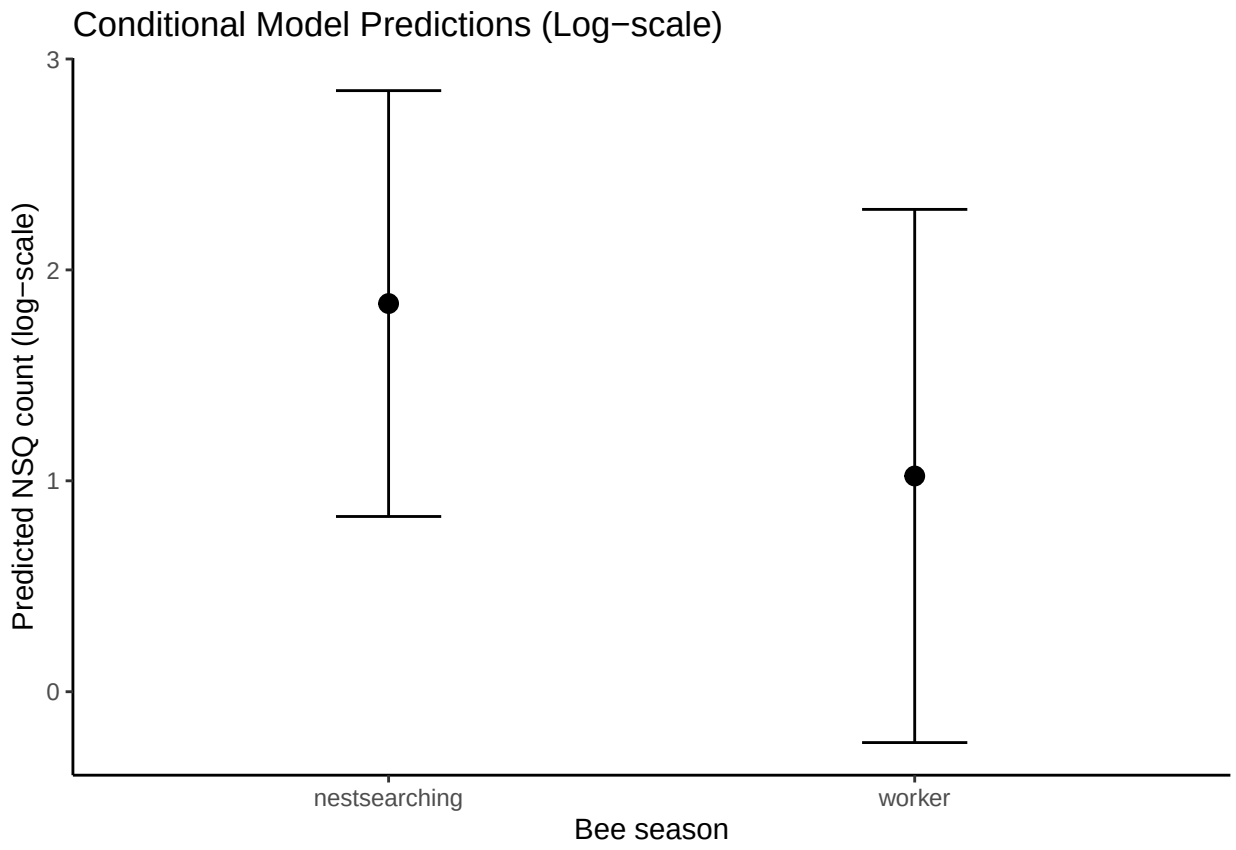


Season

Conditional model:

```
# Create newdata with discrete factor levels for bee_season_type
nsq_data_season <- data.frame(
  grass_forb_perc_scaled = mean(surveys$grass_forb_perc_scaled),
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled),
  bee_season_type = factor(c("nestsearching", "worker"), levels = levels(surveys$bee_season_type)),
  plot_type = "ditch"
)

# Get predicted zero probabilities and standard errors
pred_nsq_season_cond <- predict(nsq_glm_veg_noRE,
                           newdata = nsq_data_season,
                           type = "conditional",
                           se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_season_df <- cbind(nsq_data_season,
```

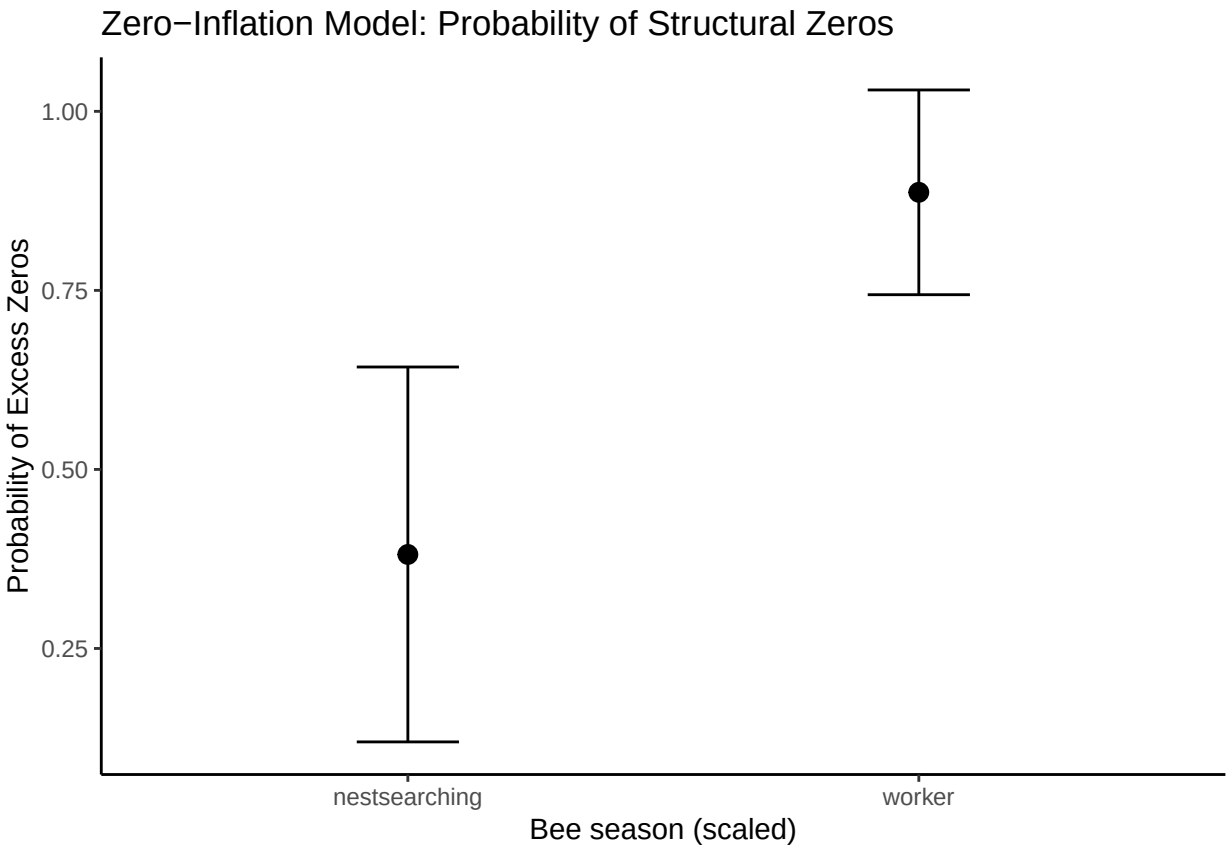
```
                cond_fit = pred_nsq_season_cond$fit,
                cond_se = pred_nsq_season_cond$se.fit)

ggplot(pred_nsq_season_df, aes(x = bee_season_type, y = cond_fit)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = cond_fit - 1.96*cond_se, ymax = cond_fit + 1.96*cond_se), width = 0.2, posit
  labs(title = "Conditional Model Predictions (Log-scale)",
       x = "Bee season",
       y = "Predicted NSQ count (log-scale)") +
  theme_classic()
```



Zero-inflated model:

```
# Get predicted zero probabilities and standard errors
pred_nsq_season_zi <- predict(nsq_glm_veg_noRE,
                              newdata = nsq_data_season,
                              type = "zprob",
                              se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_season_df <- cbind(nsq_data_season,
                zi_fit = pred_nsq_season_zi$fit,
                zi_se = pred_nsq_season_zi$se.fit)

ggplot(pred_nsq_season_df, aes(x = bee_season_type, y = zi_fit)) +
  geom_point(size = 3) +
```

```
  geom_errorbar(aes(ymin = zi_fit - 1.96*zi_se, ymax = zi_fit + 1.96*zi_se), width = 0.2, position = pos
  labs(title = "Zero-Inflation Model: Probability of Structural Zeros",
       x = "Bee season (scaled)",
       y = "Probability of Excess Zeros") +
  theme_classic()
```



Findings

Conditional model (predicts abundance of nest-searching queens at sites where queens are present). Ditch is reference plot type.

- Significant positive effect of grass+forb% on number of NSQ in ditch plots
- Marginal negative effect of plot type on # NSQ (fewer queens in field)

Zero-inflation model (predicts presence/absence of nest-searching queens).

- Significant positive effect of floral abundance in **field** plots on NSQ presence (fewer zeros when there are more flowers in the field)
- Marginal negative effect of floral abundance in ditch plots on NSQ presence (more zeros when there are more flowers along the ditch)

Workers vs. vegetation predictors

Steps

1) Try random effect structures with poisson.

- model with plot nested in site (no area) is best

```
workers_veg_full <- glmmTMB(workers ~ grass_forb_perc_scaled*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                            bee_season_type+
                            (1|area/site_id/plot_id),
                          family = poisson,
                          data = surveys)

workers_veg_noArea <- glmmTMB(workers ~ grass_forb_perc_scaled*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                              bee_season_type +
                            (1|site_id/plot_id),
                          family = poisson,
                          data = surveys)

workers_veg_siteOnly <- glmmTMB(workers ~ grass_forb_perc_scaled*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                              bee_season_type +
                            (1|site_id),
                          family = poisson,
                          data = surveys)
workers_veg_noPlot <- glmmTMB(workers ~ grass_forb_perc_scaled*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                              bee_season_type +
                            (1|area/site_id),
                          family = poisson,
                          data = surveys)

#compare models
anova(workers_veg_full, workers_veg_noArea, workers_veg_siteOnly, workers_veg_noPlot)
```

```
## Data: surveys
## Models:
## workers_veg_siteOnly: workers ~ grass_forb_perc_scaled * plot_type + bombus_floral_abun_scaled * , z
## workers_veg_siteOnly:     plot_type + bee_season_type + (1 | site_id), zi=~0, disp=~1
## workers_veg_noArea: workers ~ grass_forb_perc_scaled * plot_type + bombus_floral_abun_scaled * , zi=
## workers_veg_noArea:     plot_type + bee_season_type + (1 | site_id/plot_id), zi=~0, disp=~1
## workers_veg_noPlot: workers ~ grass_forb_perc_scaled * plot_type + bombus_floral_abun_scaled * , zi=
## workers_veg_noPlot:     plot_type + bee_season_type + (1 | area/site_id), zi=~0, disp=~1
## workers_veg_full: workers ~ grass_forb_perc_scaled * plot_type + bombus_floral_abun_scaled * , zi=~0
## workers_veg_full:     plot_type + bee_season_type + (1 | area/site_id/plot_id), zi=~0, disp=~1
##                       Df    AIC    BIC  logLik deviance  Chisq Chi Df Pr(>Chisq)
## workers_veg_siteOnly  8 327.31 354.09 -155.66   311.31
## workers_veg_noArea    9 323.00 353.13 -152.50   305.00 6.3103      1    0.01200
## workers_veg_noPlot    9 328.88 359.01 -155.44   310.88 0.0000      0    1.00000
## workers_veg_full     10 324.56 358.03 -152.28   304.56 6.3243      1    0.01191
```

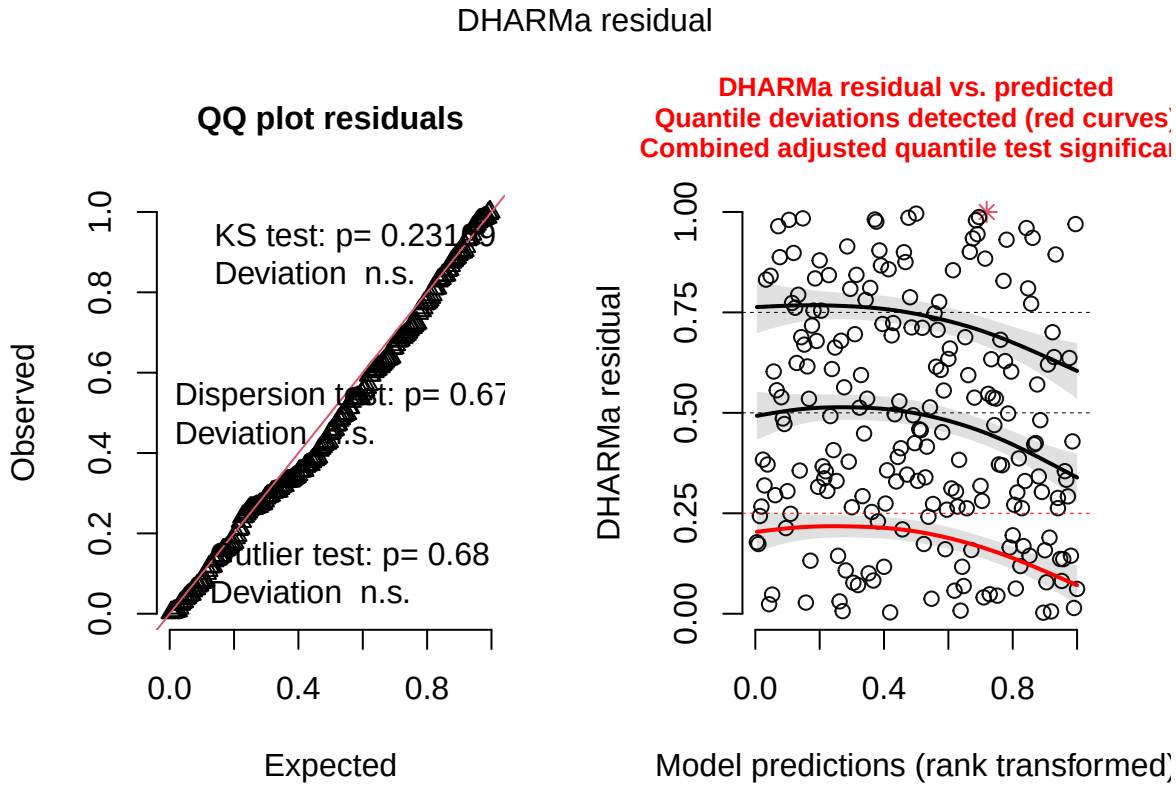
```
##
## workers_veg_siteOnly
## workers_veg_noArea   *
## workers_veg_noPlot
## workers_veg_full     *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##no area is best
```

2) Check diagnostics

- quantile test shows significant deviation

```
residuals_workers_veg_noArea <- simulateResiduals(fittedModel = workers_veg_noArea, plot = TRUE)
```

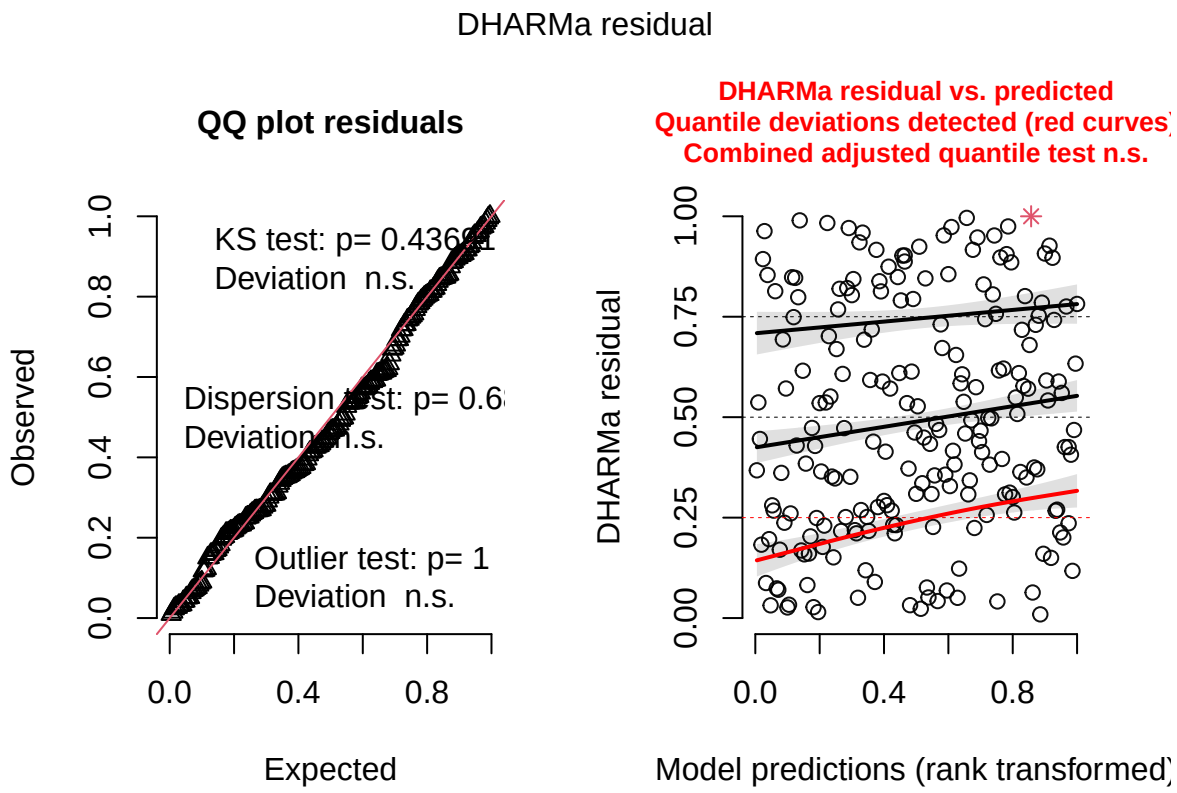


3) Try negative binomial and check diagnostics.

- diagnostic plots look good!

```
workers_veg_noArea_nbin <- glmmTMB(workers ~ grass_forb_perc_scaled*plot_type +
                                   bombus_floral_abun_scaled*plot_type +
                                   bee_season_type +
```

```
                              (1|site_id/plot_id),
                            family = nbinom2,
                            data = surveys)

#check diagnostics
#simulate residuals
residuals_workers_veg_noArea_nbin <- simulateResiduals(fittedModel = workers_veg_noArea_nbin, plot = TRU
```



4. Plot results.

```
summary(workers_veg_noArea_nbin)
```

```
##  Family: nbinom2  ( log )
## Formula:
## workers ~ grass_forb_perc_scaled * plot_type + bombus_floral_abun_scaled *
##     plot_type + bee_season_type + (1 | site_id/plot_id)
## Data: surveys
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     266.7     300.2    -123.4     246.7       200
##
## Random effects:
##
## Conditional model:
```

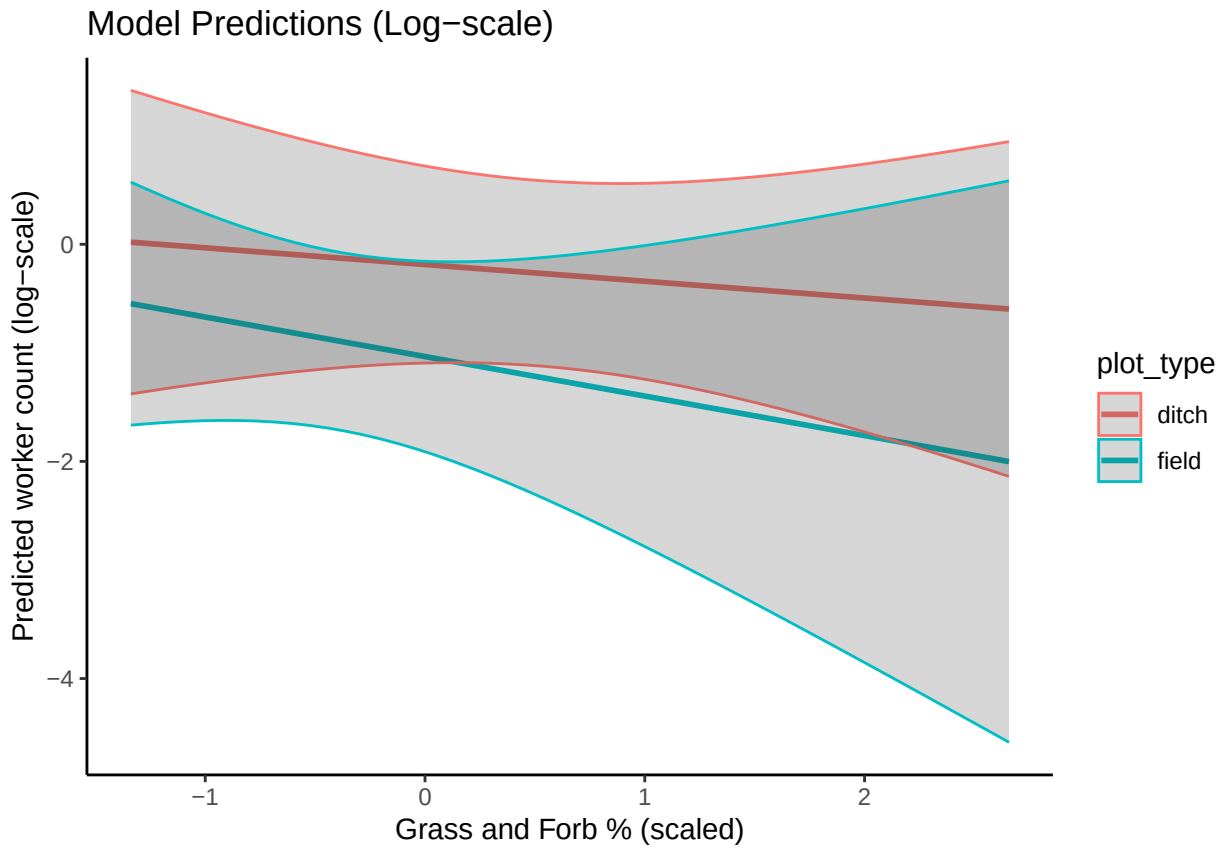
```
##  Groups          Name        Variance  Std.Dev.
##  plot_id:site_id (Intercept) 4.444e-09 6.666e-05
##  site_id         (Intercept) 1.356e-01 3.683e-01
## Number of obs: 210, groups:  plot_id:site_id, 36; site_id, 12
##
## Dispersion parameter for nbinom2 family (): 0.226
##
## Conditional model:
##                                          Estimate Std. Error z value Pr(>|z|)
## (Intercept)                               -2.6001     0.6394  -4.067 4.77e-05
## grass_forb_perc_scaled                    -0.1541     0.3054  -0.505   0.6138
## plot_typefield                            -0.8473     0.5122  -1.654   0.0981
## bombus_floral_abun_scaled                  0.3201     0.6027   0.531   0.5953
## bee_season_typeworker                      2.4132     0.5873   4.109 3.97e-05
## grass_forb_perc_scaled:plot_typefield     -0.2100     0.4953  -0.424   0.6716
## plot_typefield:bombus_floral_abun_scaled   0.4844     0.7138   0.679   0.4973
##
## (Intercept)                              ***
## grass_forb_perc_scaled
## plot_typefield                           .
## bombus_floral_abun_scaled
## bee_season_typeworker                    ***
## grass_forb_perc_scaled:plot_typefield
## plot_typefield:bombus_floral_abun_scaled
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Plots

Vegetation

```
#create new data grid
worker_data_veg<- expand.grid(
  grass_forb_perc_scaled = grass_seq,
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_veg <- predict(workers_veg_noArea_nbin,
                           newdata = worker_data_veg,
                           se.fit = TRUE,
                           re.form = NA)

# Combine predictions with data frame
pred_worker_veg_df <- cbind(worker_data_veg,
                fit = pred_worker_veg$fit,
                se = pred_worker_veg$se.fit)

ggplot(pred_worker_veg_df, aes(x = grass_forb_perc_scaled, y = fit, color = plot_type)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = fit - 1.96*se, ymax = fit + 1.96*se), alpha = 0.2) +
```

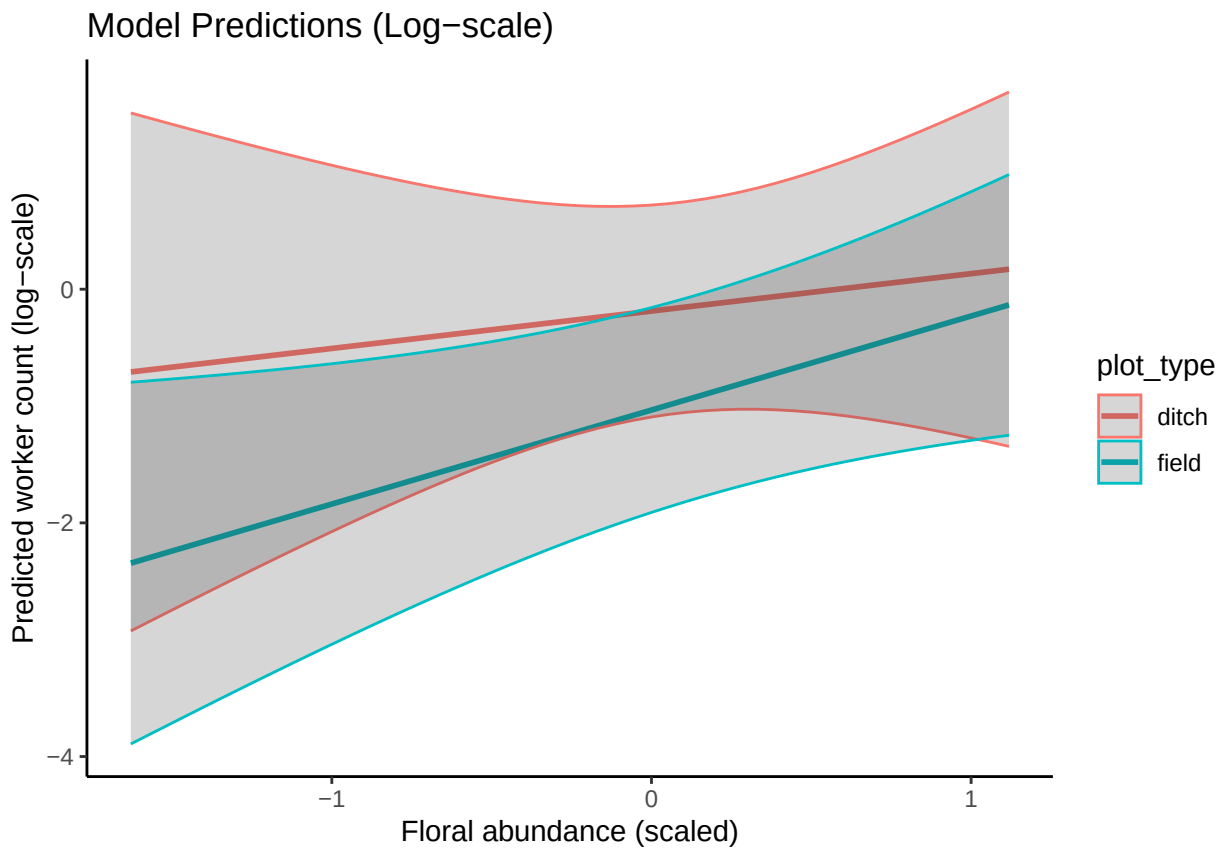
```
  labs(title = "Model Predictions (Log-scale)",
       x = "Grass and Forb % (scaled)",
       y = "Predicted worker count (log-scale)") +
  theme_classic()
```



Floral abundance

```
#create new data grid
worker_data_floral<- expand.grid(
  grass_forb_perc_scaled = mean(surveys$grass_forb_perc_scaled, na.rm = TRUE),
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = floral_seq,
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_floral <- predict(workers_veg_noArea_nbin,
                              newdata = worker_data_floral,
                              se.fit = TRUE,
                              re.form = NA)

# Combine predictions with data frame
pred_worker_floral_df <- cbind(worker_data_floral,
                 fit = pred_worker_floral$fit,
                 se = pred_worker_floral$se.fit)
```

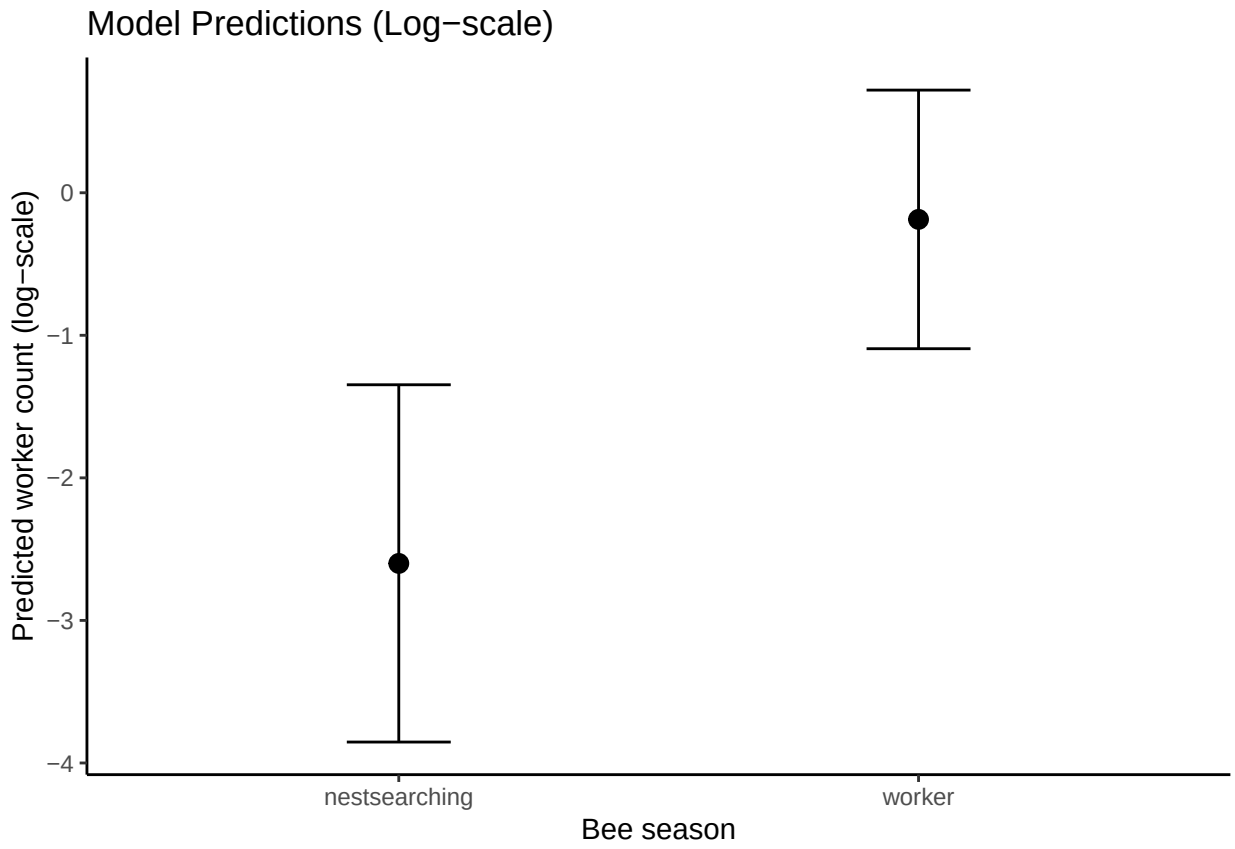
```
ggplot(pred_worker_floral_df, aes(x = bombus_floral_abun_scaled, y = fit, color = plot_type)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = fit - 1.96*se, ymax = fit + 1.96*se), alpha = 0.2) +
  labs(title = "Model Predictions (Log-scale)",
       x = "Floral abundance (scaled)",
       y = "Predicted worker count (log-scale)") +
  theme_classic()
```



Season

```
#create new data grid
worker_data_season<- expand.grid(
  grass_forb_perc_scaled = mean(surveys$grass_forb_perc_scaled, na.rm = TRUE),
  plot_type = "ditch",
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
  bee_season_type = unique(surveys$bee_season_type)
)

# Get predicted probabilities and standard errors
pred_worker_season <- predict(workers_veg_noArea_nbin,
                              newdata = worker_data_season,
                              se.fit = TRUE,
                              re.form = NA)

# Combine predictions with data frame
pred_worker_season_df <- cbind(worker_data_season,
```

```
                fit = pred_worker_season$fit,
                se = pred_worker_season$se.fit)

ggplot(pred_worker_season_df, aes(x = bee_season_type, y = fit)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = fit - 1.96*se, ymax = fit + 1.96*se), width = 0.2, position = position_dodge
  labs(title = "Model Predictions (Log-scale)",
       x = "Bee season",
       y = "Predicted worker count (log-scale)") +
  theme_classic()
```



Findings

No significant effect of any variables on worker abundance/presence.

Floral vs. plot type

Steps

1. Try random effect structures

```r
#Load packages needed for CLM
library(ordinal)
```

```
##
## Attaching package: 'ordinal'

## The following object is masked from 'package:dplyr':
##
##     slice
```

```r
library(MASS)
```

```
##
## Attaching package: 'MASS'

## The following object is masked from 'package:dplyr':
##
##     select
```

```r
surveys$bombus_floral_abun <- factor(surveys$bombus_floral_abun, ordered = TRUE)

floral_full <- clmm(as.factor(bombus_floral_abun) ~ plot_type +
                        julian.date_scaled +
                        (1|area/site_id/plot_id),
                      link = "logit",
                      data = surveys)

floral_noArea <- clmm(as.factor(bombus_floral_abun) ~ plot_type +
                        julian.date_scaled +
                        (1|site_id/plot_id),
                      link = "logit",
                      data = surveys)

#floral_plotOnly <- clmm(as.factor(bombus_floral_abun) ~ plot_type +
 #                         julian.date_scaled +
  #                        (1|plot_id),
   #                     link = "logit",
    #                    data = surveys)
##can't use since there are only 2 levels

floral_siteOnly <- clmm(as.factor(bombus_floral_abun) ~ plot_type +
                        julian.date_scaled +
                        (1|site_id),
                      link = "logit",
                      data = surveys)

floral_AreaPlot <- clmm(as.factor(bombus_floral_abun) ~ plot_type +
                        julian.date_scaled +
                        (1|area/plot_id),
                      link = "logit",
                      data = surveys)
```

```
floral_noRE <- polr(as.factor(bombus_floral_abun) ~ plot_type +
                      julian.date_scaled,
                    method = "logistic",
                    data = surveys)

#compare models AIC
AIC(floral_full, floral_noArea, floral_siteOnly, floral_AreaPlot, floral_noRE)
```

```
##                 df      AIC
## floral_full      9 665.0539
## floral_noArea    8 663.0539
## floral_siteOnly  7 661.0539
## floral_AreaPlot  8 663.3055
## floral_noRE      6 659.3055
```

```
##shows that model without random effect has best fit
```

2. Compare AIC between model with and without cover_type as a covariate.

- determined that model without cover type as covariate was better

```
floral_noRE_cover <- polr(as.factor(bombus_floral_abun) ~ plot_type +
                            cover_type +
                          julian.date_scaled,
                        method = "logistic",
                        data = surveys)

AIC(floral_noRE_cover, floral_noRE)
```

```
##                   df      AIC
## floral_noRE_cover  7 661.2522
## floral_noRE        6 659.3055
```

```
##not including cover is better
```

3. Try including julianDate * plot_type

- determined that model with interaction term was better

```
floral_noRE_interaction <- polr(as.factor(bombus_floral_abun) ~ plot_type*julian.date_scaled,
                        method = "logistic",
                        data = surveys)
AIC(floral_noRE_interaction, floral_noRE)
```

```
##                          df      AIC
## floral_noRE_interaction  7 649.2044
## floral_noRE              6 659.3055
```

```
##interaction is better
```

4. Look at output

```
summary(floral_noRE_interaction)
```

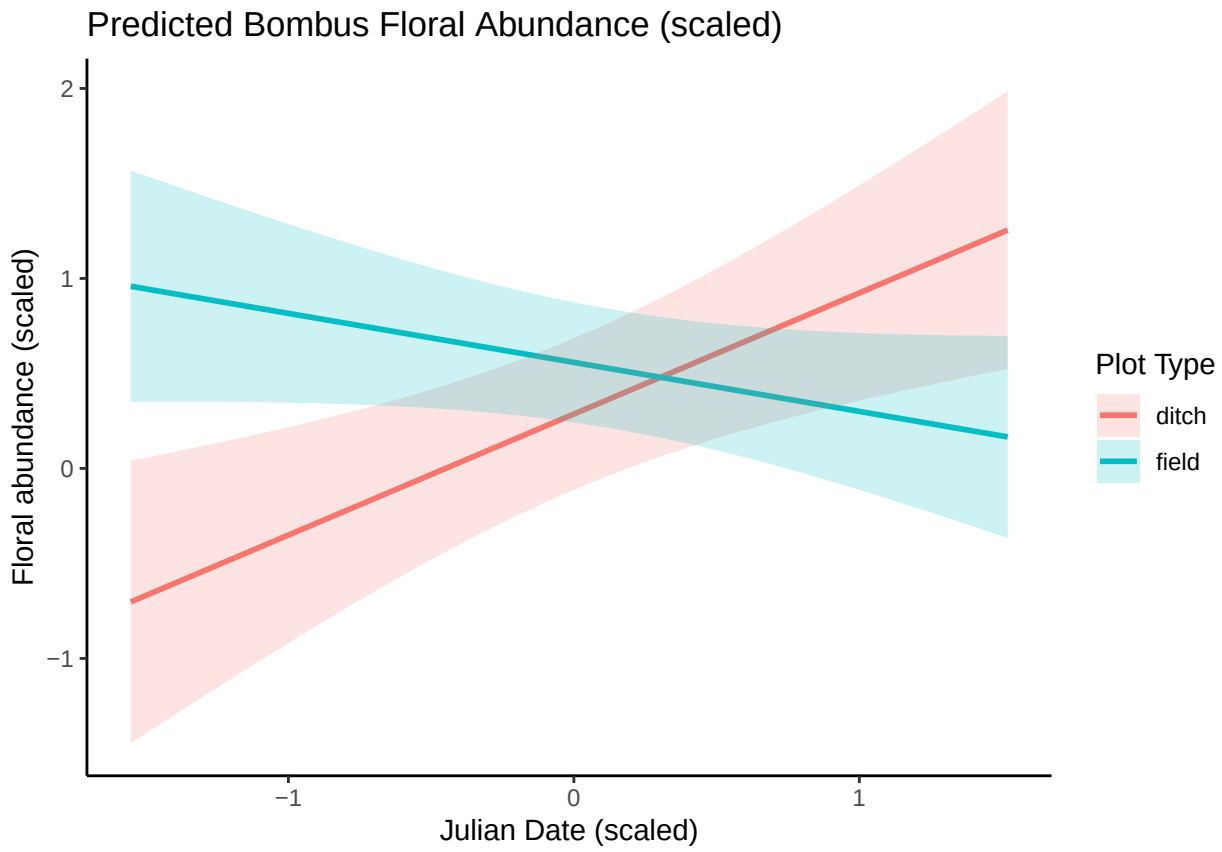
```
##
## Re-fitting to get Hessian

## Call:
## polr(formula = as.factor(bombus_floral_abun) ~ plot_type * julian.date_scaled,
##     data = surveys, method = "logistic")
##
## Coefficients:
##                                     Value Std. Error t value
## plot_typefield                     0.2718     0.2569   1.058
## julian.date_scaled                 0.6371     0.2059   3.094
## plot_typefield:julian.date_scaled -0.8951     0.2611  -3.428
##
## Intercepts:
##     Value   Std. Error t value
## 0|1 -1.3685  0.2391    -5.7243
## 1|2 -0.8672  0.2224    -3.8987
## 2|3  0.0166  0.2118     0.0783
## 3|4  1.0741  0.2261     4.7504
##
## Residual Deviance: 635.2044
## AIC: 649.2044
```

Plots

```
# Sequence of Julian dates
date_seq <- seq(
  from = min(surveys$julian.date_scaled, na.rm = TRUE),
  to = max(surveys$julian.date_scaled, na.rm = TRUE),
  length.out = 50
)

# Get predicted probabilities per category with CIs
pred_floral_veg <- emmeans(
  floral_noRE_interaction,
  ~ julian.date_scaled * plot_type,
  at = list(julian.date_scaled = date_seq),
  type = "response",  # To get probabilities
)
```

```
##
## Re-fitting to get Hessian
```

```r
# Convert to dataframe
pred_floral_veg_df <- as.data.frame(pred_floral_veg)

#plot
ggplot(pred_floral_veg_df, aes(x = julian.date_scaled, y = response, color = plot_type)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = asymp.LCL, ymax = asymp.UCL, fill = plot_type), alpha = 0.2, color = NA) +
  labs(
    title = "Predicted Bombus Floral Abundance (scaled)",
    x = "Julian Date (scaled)",
    y = "Floral abundance (scaled)",
    color = "Plot Type",
    fill = "Plot Type"
  ) +
  theme_classic()
```



Findings

- Julian date is a significant predictor: floral abundance tends to increase as season progresses.
- The effect of Julian date changes depending on plot type (significant interaction).
  - In the ditch, floral abundance increases throughout the season.
  - In the field, floral abundance decreases throughout the season.
- Plot type alone has no strong main effect but modifies the Julian date effect.

Rodent burrows vs. vegetation predictors

Steps

1) Try random effect structures with poisson.

- model with site only is best

```
rodent_veg_areaOnly <- glmmTMB(num_burrows ~ cover_type*plot_type +
                                 bombus_floral_abun_scaled*plot_type +
                               (1|area),
                             family = poisson,
                             data = surveys)

rodent_veg_siteOnly <- glmmTMB(num_burrows ~ cover_type*plot_type +
                                 bombus_floral_abun_scaled*plot_type +
                               (1|site_id),
                             family = poisson,
                             data = surveys)
rodent_veg_siteArea <- glmmTMB(num_burrows ~ cover_type*plot_type +
                                 bombus_floral_abun_scaled*plot_type +
                               (1|area/site_id),
                             family = poisson,
                             data = surveys)

#compare models
anova(rodent_veg_areaOnly, rodent_veg_siteOnly, rodent_veg_siteArea)
```

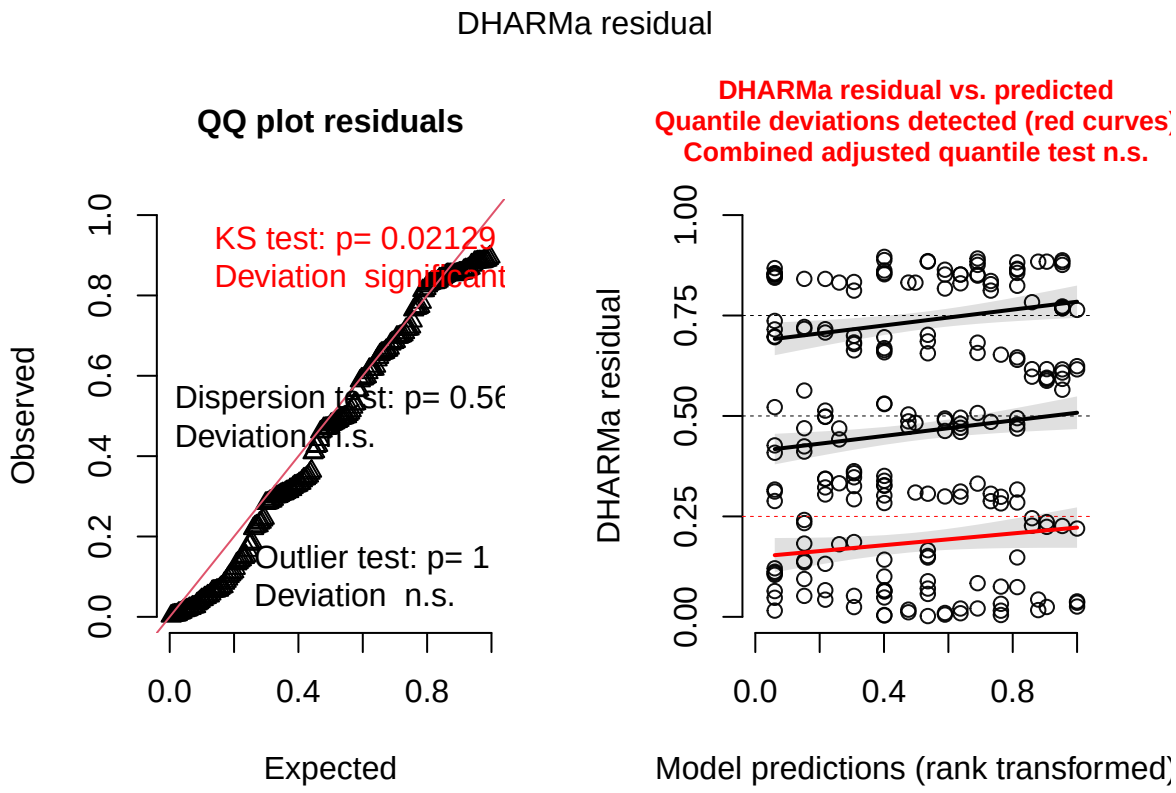
```
## Data: surveys
## Models:
## rodent_veg_areaOnly: num_burrows ~ cover_type * plot_type + bombus_floral_abun_scaled * , zi=~0, disp
## rodent_veg_areaOnly:     plot_type + (1 | area), zi=~0, disp=~1
## rodent_veg_siteOnly: num_burrows ~ cover_type * plot_type + bombus_floral_abun_scaled * , zi=~0, disp
## rodent_veg_siteOnly:     plot_type + (1 | site_id), zi=~0, disp=~1
## rodent_veg_siteArea: num_burrows ~ cover_type * plot_type + bombus_floral_abun_scaled * , zi=~0, disp
## rodent_veg_siteArea:     plot_type + (1 | area/site_id), zi=~0, disp=~1
##                      Df    AIC    BIC  logLik deviance  Chisq Chi Df Pr(>Chisq)
## rodent_veg_areaOnly  7 4454.1 4477.6 -2220.1   4440.1
## rodent_veg_siteOnly  7 3381.0 3404.5 -1683.5   3367.0 1073.1      0     <2e-16
## rodent_veg_siteArea  8 3382.9 3409.7 -1683.5   3366.9    0.1      1     0.7518
##
## rodent_veg_areaOnly
## rodent_veg_siteOnly ***
## rodent_veg_siteArea
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##site only is best
```

2) Check diagnostics

- KS test shows significant deviation

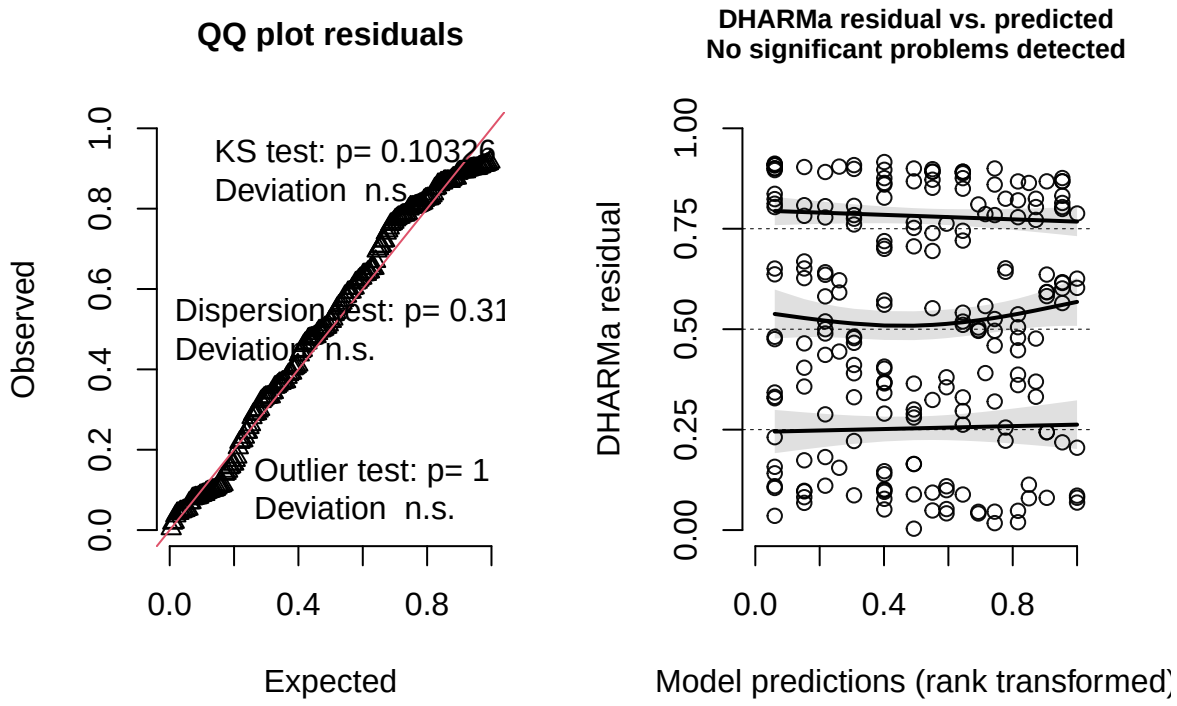
```
residuals_rodent_veg_siteOnly <- simulateResiduals(fittedModel = rodent_veg_siteOnly, plot = TRUE)
```



3) Try negative binomial and check diagnostics.

- diagnostic plots look good!

```
rodent_veg_siteOnly_nbin <- glmmTMB(num_burrows ~ cover_type*plot_type +
                                      bombus_floral_abun_scaled*plot_type +
                                    (1|site_id),
                                  family = nbinom2,
                                  data = surveys)

#check diagnostics
#simulate residuals
residuals_rodent_veg_siteOnly_nbin <- simulateResiduals(fittedModel = rodent_veg_siteOnly_nbin, plot = T
```



4. Plot results.

```
summary(rodent_veg_siteOnly_nbin)
```

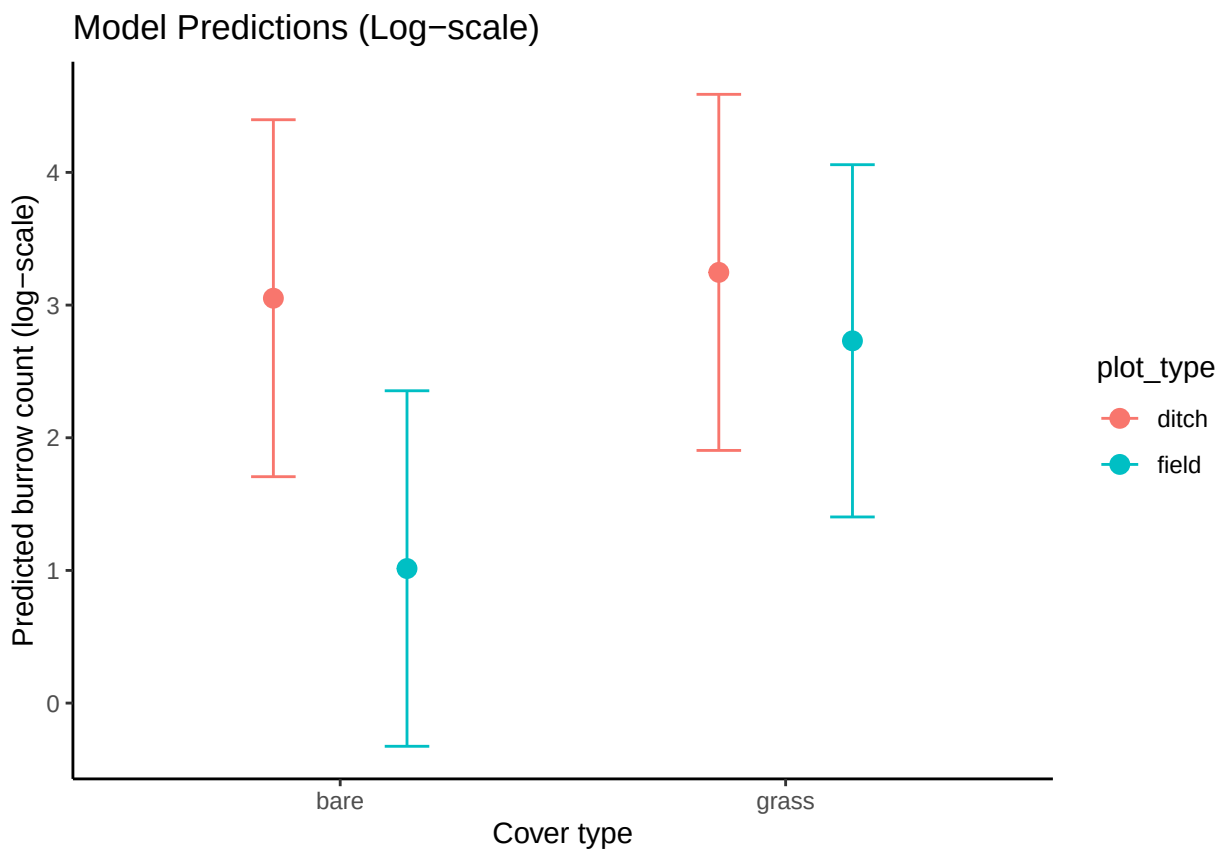
```
##  Family: nbinom2  ( log )
## Formula:
## num_burrows ~ cover_type * plot_type + bombus_floral_abun_scaled *
##     plot_type + (1 | site_id)
## Data: surveys
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    1487.2    1514.0    -735.6    1471.2       202
##
## Random effects:
##
## Conditional model:
##  Groups  Name        Variance Std.Dev.
##  site_id (Intercept) 2.679    1.637
## Number of obs: 210, groups:  site_id, 12
##
## Dispersion parameter for nbinom2 family ():  1.8
##
## Conditional model:
##                                      Estimate Std. Error z value Pr(>|z|)
## (Intercept)                           3.05177    0.68639   4.446 8.74e-06
```

```
## cover_typegrass                             0.19475    0.96941   0.201    0.841
## plot_typefield                             -2.03756    0.20960  -9.721  < 2e-16
## bombus_floral_abun_scaled                   0.09791    0.11273   0.869    0.385
## cover_typegrass:plot_typefield              1.52157    0.27992   5.436 5.46e-08
## plot_typefield:bombus_floral_abun_scaled -0.16847    0.13241  -1.272    0.203
##
## (Intercept)                                ***
## cover_typegrass
## plot_typefield                             ***
## bombus_floral_abun_scaled
## cover_typegrass:plot_typefield             ***
## plot_typefield:bombus_floral_abun_scaled
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Plots

Vegetation

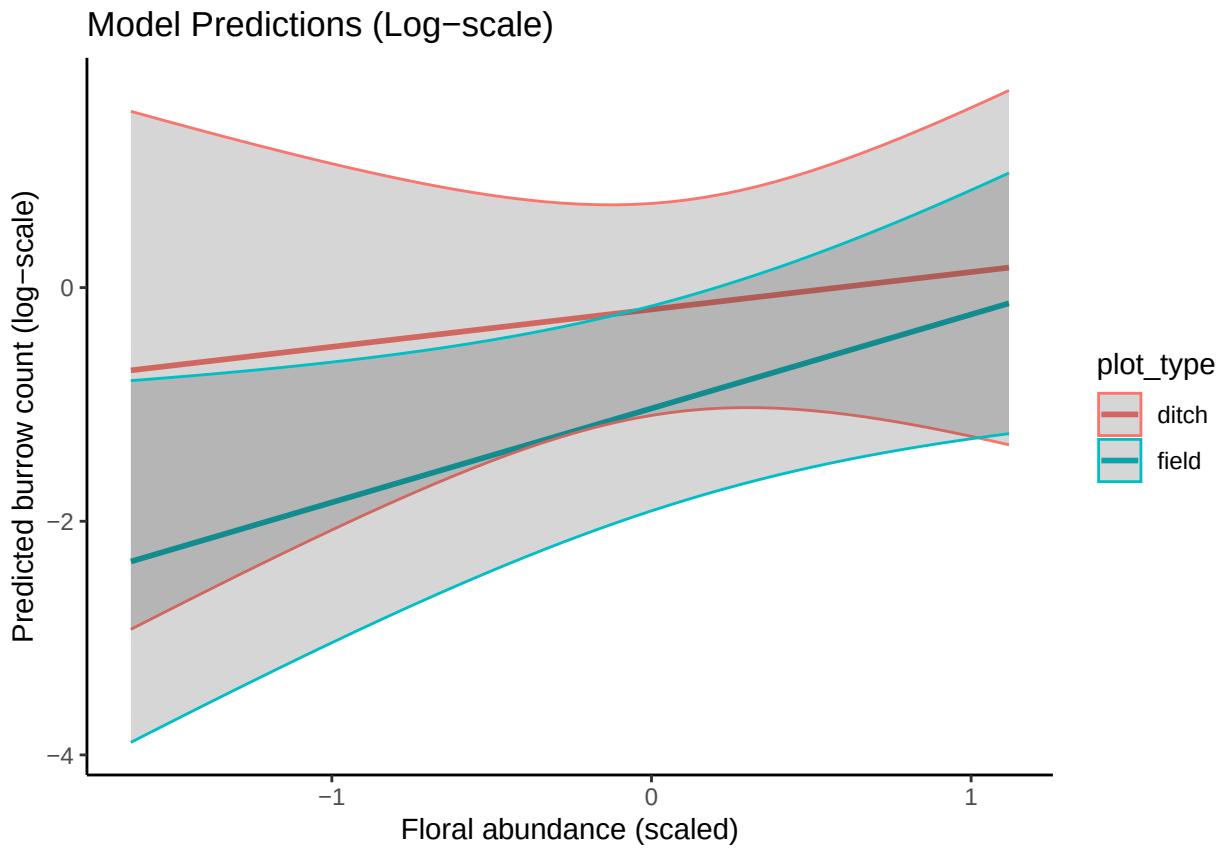
```
#create new data grid
rodent_data_veg<- expand.grid(
  cover_type = unique(surveys$cover_type),
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE)
)

# Get predicted probabilities and standard errors
pred_rodent_veg <- predict(rodent_veg_siteOnly_nbin,
                           newdata = rodent_data_veg,
                           se.fit = TRUE,
                           re.form = NA)

# Combine predictions with data frame
pred_rodent_veg_df <- cbind(rodent_data_veg,
                fit = pred_rodent_veg$fit,
                se = pred_rodent_veg$se.fit)

# Plot conditional predictions
ggplot(pred_rodent_veg_df, aes(x = cover_type, y = fit, color = plot_type)) +
  geom_point(size=3, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = fit - 1.96*se, ymax = fit + 1.96*se), width = 0.2, position = position_dodge
  labs(title = "Model Predictions (Log-scale)",
       x = "Cover type",
       y = "Predicted burrow count (log-scale)") +
  theme_classic()
```



Floral abundance

```
#create new data grid
rodent_data_floral<- expand.grid(
  cover_type = "grass",
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = floral_seq
)

# Get predicted probabilities and standard errors
pred_rodent_floral <- predict(rodent_veg_siteOnly_nbin,
                              newdata = rodent_data_floral,
                              se.fit = TRUE,
                              re.form = NA)

# Combine predictions with data frame
pred_rodent_floral_df <- cbind(rodent_data_floral,
                 fit = pred_rodent_floral$fit,
                 se = pred_rodent_floral$se.fit)

ggplot(pred_worker_floral_df, aes(x = bombus_floral_abun_scaled, y = fit, color = plot_type)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = fit - 1.96*se, ymax = fit + 1.96*se), alpha = 0.2) +
  labs(title = "Model Predictions (Log-scale)",
       x = "Floral abundance (scaled)",
       y = "Predicted burrow count (log-scale)") +
```

```
  theme_classic()
```



Findings

At field plots, there are more rodent burrows in grassy sites than in bare sites. No siginficant effect of floral abundance on rodent burrows in either plot type.